

Here is a **clean, ready-to-use mapping table** showing **OLD frontend API endpoints → NEW endpoints** (after refactor), including **HTTP method changes** and **new request body format** when applicable.

 ADMIN API — Endpoint Migration Table

Old Endpoint	Old Method	New Endpoint	New Method	New Rec
/api/admin/lots	GET	/api/admin/lots	GET	—
/api/admin/lots	POST	/api/admin/lots	POST	{ name, address, number_c
/api/admin/lots/<id>	PUT	/api/admin/lots/<id>	PUT	Optional address, number_c
/api/admin/lots/<id>	DELETE	/api/admin/lots/<id>	DELETE	—
/api/admin/slots/<id>	DELETE	/api/admin/slots/<id>	DELETE	—
/api/admin/users	GET	Merged into /api/admin/reports	POST	{ "view mobile?,"
/api/admin/reservations	GET	/api/admin/reports	POST	{ "view user_id? active_c
/api/admin/summary	GET	/api/admin/reports	POST	{ "view
/api/admin/search	GET	/api/admin/search	GET	Same: ? type=...
/api/admin/reports/occupancy	GET	/api/admin/reports	POST	{ "view
/api/admin/reports/revenue	GET	/api/admin/reports	POST	{ "view

Old Endpoint	Old Method	New Endpoint	New Method	New Rec
/api/admin/reports/reservations	GET	/api/admin/reports	POST	{ "view "reserva
/api/admin/reports/pdf	POST	/api/admin/reports/pdf	POST	{ chart
/api/admin/reminders/logs	GET	/api/admin/reports	POST	{ "view page?, p user_id? date_to?

✓ USER API — Endpoint Migration Table

Good news: **user routes remain almost identical**, except for the `SpotActivityResource` GET views.

Old Endpoint	Old Method	New Endpoint	New Method	Notes
/api/user/profile/<id>	GET	/api/user/profile/<id>	GET	Same
/api/user/spots (book/release)	POST	/api/user/spots	POST	Body: `{ action: "book"}`
/api/user/spots (reservations list)	GET	/api/user/spots? view=reservations	GET	Same logic
/api/user/spots? view=summary	GET	Same	GET	Summary view
/api/user/spots? view=monthly	GET	Same	GET	Monthly report
/api/user/spots? view=location	GET	Same	GET	Location report
/api/user/spots? view=recent	GET	Same	GET	Last 5
/api/user/spots? view=activity	GET	Same	GET	Recent activity

Old Endpoint	Old Method	New Endpoint	New Method	Notes
<code>/api/user/lots?pin_code=...</code>	GET	<code>/api/user/lots?pin_code=...</code>	GET	Same
<code>/api/user/pincodes</code>	GET	<code>/api/user/pincodes</code>	GET	Same
<code>/api/user/reservations</code>	GET	<code>/api/user/reservations</code>	GET	Same

★ Quick Reference: New Admin Report Views

These are the valid POST views under:

```
bash
```

```
POST /api/admin/reports
```

View	Purpose	Example Body
<code>summary</code>	Dashboard top numbers	<code>{ "view": "summary" }</code>
<code>occupancy</code>	Available vs occupied	<code>{ "view": "occupancy" }</code>
<code>revenue</code>	Revenue per lot per month	<code>{ "view": "revenue" }</code>
<code>reservations_by_lot</code>	Booking count per lot	<code>{ "view": "reservations_by_lot" }</code>
<code>reservations</code>	Detailed reservations	<code>{ "view": "reservations", "lot_id": 2 }</code>
<code>users</code>	User list with filters	<code>{ "view": "users", "email": "abc" }</code>
<code>reminder_logs</code>	Reminder job logs	<code>{ "view": "reminder_logs", "page": 1 }</code>

If you want, I can also produce:

- ✓ Vue.js API service wrappers for all new endpoints
- ✓ Axios request examples
- ✓ A diff report showing exact frontend-breakpoints
- ✓ Migration checklist for your admin pages

Just tell me what you want next.